Lecture 6
Bash scripting

Comp 206 – Introduction to Software Systems – Fall 2026

Mathieu Bérubé-Vallières

McGill University

September 15, 2026

External announcements

Bérubé-Vallières © 2026
See last slide for complete credits


GAMEDEV
MCGILL
PROGRAMMER? ARTIST? DESIGNER? MUSICIAN?
COMPLETE BEGINNER? JOIN GDM TODAY!
RECRUITMENT DEADLINE
OCTOBER 2ND
JOIN THE RECRUITMENT
DISCORD SERVER!
GAMEDEV.MENTOR@SSMU.CA
GAMEDEVMCGILL.CA
@GAMEDEVMCGILL

End of lecture 5: Permissions

Bérubé-Vallières © 2026
See last slide for complete credits

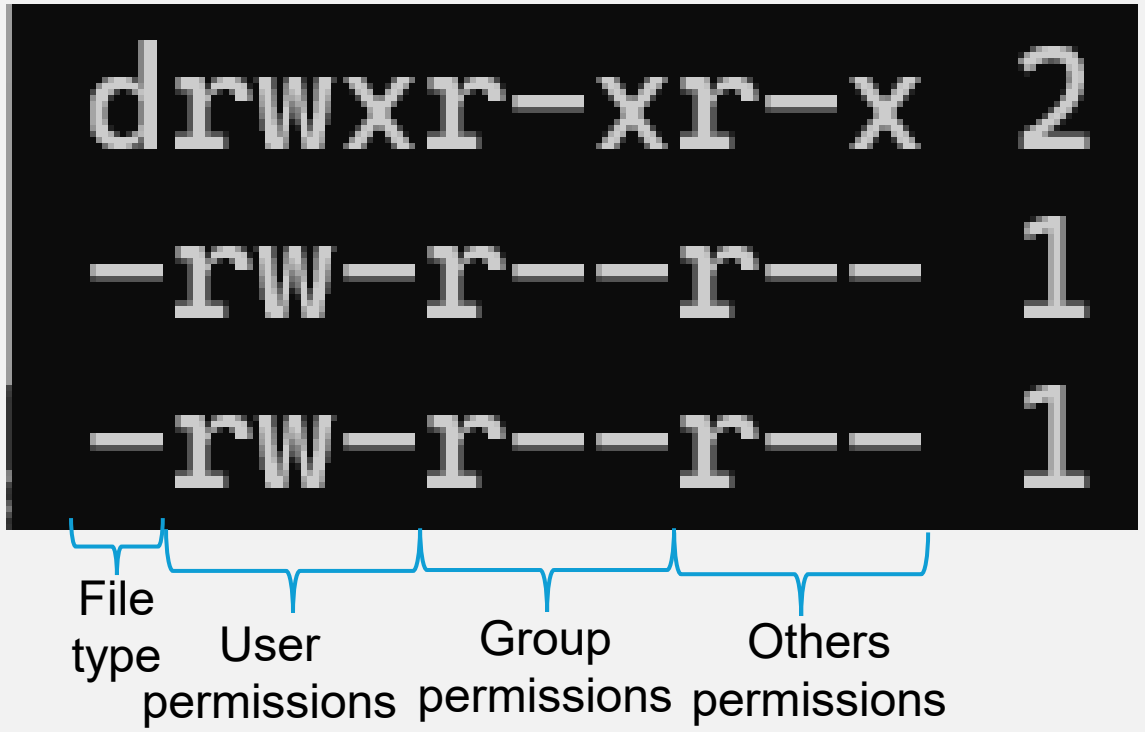
Last class: Permissions

- All files are **owned** by a user and a group.
  - Usually, this owner is the user that created the file.
- Permissions on files exists at three level (classes):
  - **user**, **group** and **others**.
- Three types of rights can be given:
  - **read**, **write** and **execute**.
- Any combination of these rights must be given to these three levels.

Bérubé-Vallières © 2026
See last slide for complete credits

Last class: Symbolic notation

- The symbolic notation for permissions is displayed as a string of 9 characters (10 if including file type)
  - (0th : indicates if the file is a directory.)
  - 1st : indicates if the user that owns the file has read access to the file.
  - 2nd : indicates if the user that owns the file has write access to the file.
  - 3rd : indicates if the user that owns the file has execute access to the file.
  - 4th , 5th , 6th : indicates if the group owner has read, write or execute.
  - 7th , 8th, 9th : indicates if all other users have read, write or execute.



Last class: `chmod` command

- **Ch**ange **mod**e: Change file permissions
  - Can be used by owner or superuser (root)
- Who:
  - u : The user who owns the file (this means “you.”)
  - g : The group the file belongs to.
  - o : The other users
  - a : all of the above (an abbreviation for ugo)
- Permission
  - r : Permission to read the file.
  - w : Permission to write (or delete) the file.
  - x : Permission to execute the file (for directories, this is needed to enter them)
- Changes to
  - = : become
  - + : add
  - - : remove

Bérubé-Vallières © 2026
See last slide for complete credits

Last class: `chmod` command examples

- The syntax of the command is as follows:
  - `chmod who=permission files`

Here are a few examples of the chmod command:

- Give read permission to group
  - `chmod g+r file.txt`
- Give write/execute permission to you (user)
  - `chmod u+wx file2.txt`
- Remove all permissions from others
  - `chmod o= file3.txt`
- Give read/write permission only to user and group
  - `chmod ug=rw file4.* file2.txt`
- Give read only to user and execute only to group
  - `chmod u=r,g=x file4.* file2.txt` (no space after the comma)

Bérubé-Vallières © 2026
See last slide for complete credits

Binary representation of permissions

- Given a 9 character permission string:

rwx --- rw-

- We can equivalent represent it in binary

111 000 110

- where 1 means “on” and 0 means “off”

Example: What does 010 111 001 represent?

Bérubé-Vallières © 2026
See last slide for complete credits

Binary representation of permissions

- Given a 9 character permission string:

rwx --- rw-

- We can equivalent represent it in binary

111 000 110

- where 1 means “on” and 0 means “off”

Example: What does 010 111 001 represent?

- Answer: -w- rwx --x

Bérubé-Vallières © 2026
See last slide for complete credits

Octal representation of permissions

- Given the binary representation of permissions, we can convert it to *octal* (base 8) by converting each group of 3 bits to its corresponding digit as follows:

Binary	Octal
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Octal representation of permissions

- For example, given

111 000 110

- The octal representation is

706

Binary	Octal
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Octal representation of permissions

- For example, given

111 000 110

- The octal representation is

706

Binary	Octal	Symbolic
000	0	`---`
001	1	`--x`
010	2	`-w-`
011	3	`-wx`
100	4	`r--`
101	5	`r-x`
110	6	`rw-`
111	7	`rwx`

`chmod` and octal representation

- `chmod` supports octal representation
  - Often used when want to set up all permissions at the same time
- Ex:
  - chmod 400 filename
    - Give user permission to read and no permissions to group and others
  - chmod 644 filename
    - Give user permission to read and write and only give permission to read to group and others
  - chmod 700 filename
    - Give user all permissions and no permissions to group and others

Binary	Octal	Symbolic
000	0	`---`
001	1	`--x`
010	2	`-w-`
011	3	`-wx`
100	4	`r--`
101	5	`r-x`
110	6	`rw-`
111	7	`rwx`

Bérubé-Vallières © 2026
See last slide for complete credits

Bash scripting

Bérubé-Vallières © 2026
See last slide for complete credits

Bash scripting

- So far, we have used Bash in *interactive* mode
  - Commands are read from command-line
- The Bash shell supports the execution of commands from files (non-interactive mode)
  - Lets user or system automate common sequences of commands
- These files are called *scripts*
- Scripts are run in a similar way to any programming language
  - Scripting languages are *interpreted* (as opposed to *compiled*): code is executed directly without being compiled to machine code

Bérubé-Vallières © 2026
See last slide for complete credits

Example

```
#!/bin/bash
mkdir my_dir
cd my_dir
touch my_file.txt
ls
```



Tells the OS that this is a bash file
(more on this later)

Bérubé-Vallières © 2026
See last slide for complete credits

Shebang (1/2)

- File type is not determined by file extension in Unix
  - Convention is that Bash scripts use file extension .sh, but this is only a visual indicator for humans!
- To run a Bash script (for example, `comp206/my_script`), one way would be to give the script as input to the Bash interpreter:
  - `bash comp206/my_script`
  - Compare: If `comp206/my_script` was a python3 program instead, we would use
    - `python3 comp206/my_script`
  - This is not how we usually run Bash scripts!

Bérubé-Vallières © 2026
See last slide for complete credits

Shebang (2/2)

- We can also tell the OS which interpreter the file should be run with by using a *shebang*
- The shebang is the very first line of the file: It consists of a hash symbol, followed by an exclamation mark, followed by the path to the interpreter.
- For Bash, this is: `#!/bin/bash`
  - Compare: for python3, this would be `#!/bin/python3`
- Can now directly *execute* the file, and the OS will select the correct interpreter
  - i.e. can now simply type `comp206/my_script` instead of `bash comp206/my_script`
- All of your scripts should be run this way!

Bérubé-Vallières © 2026
See last slide for complete credits

Running a script

1. Give yourself execute permissions on the script
2. Type the path of the script to execute it, followed by the flags and arguments
   - e.g. `comp206/myprogram.sh arg1 arg2`

- If the script is in the current directory, you must explicitely specify that it is in the current directory by using `./` at the front
  - e.g. `./myprogram.sh arg1 arg2`

Bérubé-Vallières © 2026
See last slide for complete credits

Why ./ in front?

- When typing the name of a program, the shell finds the programs as follows:
  - If the name contains /, the shell assumes that this is a path (could be relative or absolute) and looks for the program at that location
  - If the name doesn't contain /, the shell looks in all the folders listed in a shell variable called PATH (more on that in later class)
    - try `echo $PATH`
    - For example, when running `ls`, the shell finds that command in `/usr/bin/ls`
- Looking at the rules above, this implies that we are unable to refer to a program in the current directory (and not in PATH) by using a relative path without /
  - This is by design: What if someone maliciously named a virus "ls"

Bérubé-Vallières © 2026
See last slide for complete credits

When are scripts used? (Examples)

- Login scripts (Run automatically at login)
  - Write scripts to help you get to where you want to go
  - Write scripts to customize the environment
- During development and testing (Run manually during session)
  - Write scripts that help you to
    - Compile quickly and manage errors and executing the program
    - Copying to and from repository
    - Making your own local backups
    - Help run testing (repeated tasks, good candidate for automation)
- Logout scripts (Run automatically at logout)
  - Write scripts to do housekeeping
    - Automating backup procedures
    - Automating the logging of events
    - Automating the deletion of files (empty trash)

Bérubé-Vallières © 2026
See last slide for complete credits

Example: Writing a script

Want to make a script that will move all my txt files to a directory called textfiles and all my doc files to a directory called docfiles

```
$ vim move.sh
```

(vim)

```
#!/bin/bash
# Move txt and doc files from curr dir
# to new dirs txtfiles and docfiles
mkdir txtfiles
mkdir docfiles
mv *.txt txtfiles
mv *.doc docfiles
```

shebang

comments

```
$ chmod +x move.sh
$ ./move.sh
```


Bérubé-Vallières © 2026
See last slide for complete credits

Example: Writing a script

Want to make a script that will move all my txt files to a directory called textfiles and all my doc files to a directory called docfiles

```
$ vim move.sh
```

(vim)

```
#!/bin/bash
# Move txt and doc files from curr dir
# to new dirs txtfiles and docfiles
mkdir txtfiles
mkdir docfiles
mv *.txt txtfiles
mv *.doc docfiles
```

shebang

comments

```
$ chmod +x move.sh
$ ./move.sh
```

(Could make this script better by checking if dir already exists (right now mkdir will output a warning): More on this next class)

Bérubé-Vallières © 2026
See last slide for complete credits

Variables

- Variables are assigned using the syntax `name=value`
  - Important: no space before and after equal sign!
- To use a variable, `$name` is used
- Examples:
  - str="hello world"
  - num=5
  - date=$(date) # what does this do?
  - echo $str
  - echo $num
  - echo $date

Bérubé-Vallières © 2026
See last slide for complete credits

Positional variables

- Parameters passed to a bash script (e.g. on the command line) are numbered from $1
- For example, if we run `./script 5 2 Bob`
- From the script we can access `5, 2, Bob` using $1, $2, $3

Bérubé-Vallières © 2026
See last slide for complete credits

Example

- The following script takes as input a file extension and a directory, and moves all files with that extension from the current directory to the specified directory

```
#!/bin/bash
mv *.$1 $2
```


Bérubé-Vallières © 2026
See last slide for complete credits

Example

- The following script takes as input a file extension and a directory, and moves all files with that extension from the current directory to the specified directory

```
#!/bin/bash
mv *.$1 $2
```

This script assumes the user entered the expected arguments: Will learn how to check user input next class!

Bérubé-Vallières © 2026
See last slide for complete credits

Next class

- More on variables
- Control structures (if/else, loops)

Bérubé-Vallières © 2026
See last slide for complete credits

Additional credits

- Some content from this lecture is based on:
  - Past Comp 206 slides from Prof. Vybihal

Bérubé-Vallières © 2026
See last slide for complete credits